SWEN 221

Marco Servetto
CO258
www.youtube.com/MarcoServetto


The ugly parts of
old Java


The ugly parts of old Java
Subtyping
Coercion

The ugly parts of old Java

Overloading

Overriding

The two brothers


We start discussing
Subtyping vs Coercion

Subtyping recap

- In Java, we can write the following:

```
void g(Object y) { … }
void f(String x) { g(x); }
```

  - This is OK because a String is always a valid Object
  - But, this **does not** compile:

```
void g(String y) { … }
void f(Object x) { g(x); }
```

  - Because an Object is not always a valid String.
    - E.g. an Integer is not a String

- We say String **is a subtype of** Object
  - denoted by **`String <: Object`**
  - A subtype can be used whenever its supertype(s) are expected

Inheritance and Subtyping

- For two classes/interfaces A and B:
  - if A **extends** B, or A **implements** B, then A <: B

```
class Point { int x; int y; … }
class ColouredPoint extends Point { int colour; }

void move(Point p, int dx, int dy) {
 p.x += dx;
 p.y += dy;
}
...
ColouredPoint cp = new ColouredPoint(…);
move(cp,1,1);
System.out.println("cp.x = " + cp.x);
```

Through p we cannot see “colour” but it is there!

  - Therefore, in this code, ColouredPoint <: Point
  - Meaning we can use a ColouredPoint instead of a Point!

Static vs Dynamic Typing

- **What is it?**
  - **Static Type** – the declared type of a variable in the code
  - **Dynamic Type** – the actual type of an object in a moment of the execution

```
class Point { int x; int y; … }
class ColouredPoint extends Point { int colour; }

void move(Point p, int dx, int dy) {
 p.x += dx;
 p.y += dy;
}
...
ColouredPoint cp = new ColouredPoint(…);
move(cp,1,1);
System.out.println("cp.x = " + cp.x);
```

  - Here, parameter p has **static type** Point
  - But, in the shown call, p refers to object with **dynamic type** ColouredPoint
  - Can only access fields/methods through static type of p

Properties of Subtyping

- Subtyping properties:
  - Transitive
    - If X <: Y and Y <: Z then X <: Z
  - Reflexive
    - X <: X always holds!

```
class Point { int x; int y; … }
class ColouredPoint extends Point { int colour; }
class Coloured3DPoint extends ColouredPoint { int z; }
```

- So, does Coloured3DPoint <: Point hold?

Exercise – which ones work?

```
class Point { int x; int y; … }
class Point3D extends Point { int z; }
class ColouredPoint extends Point { int colour; }
class ColouredPoint3D extends ColouredPoint { int z; }

void move(Point p) { … }
void paint(ColouredPoint cp) { … }

ColouredPoint3D cp3= new ColouredPoint3D(…);
Point3D p3= new Point3D(…);
```

A) `move(cp3);` B) `move(p3);` C) `paint(p3);`

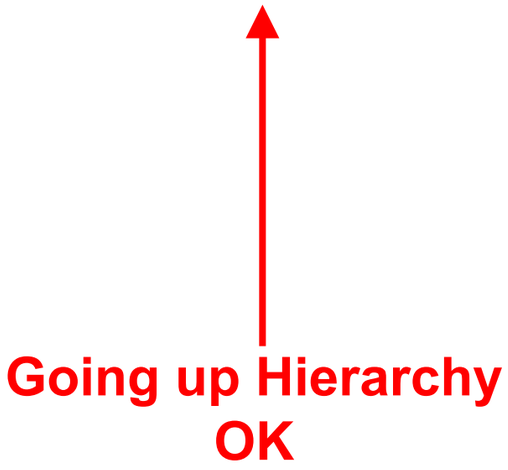
Exercise – which ones work?

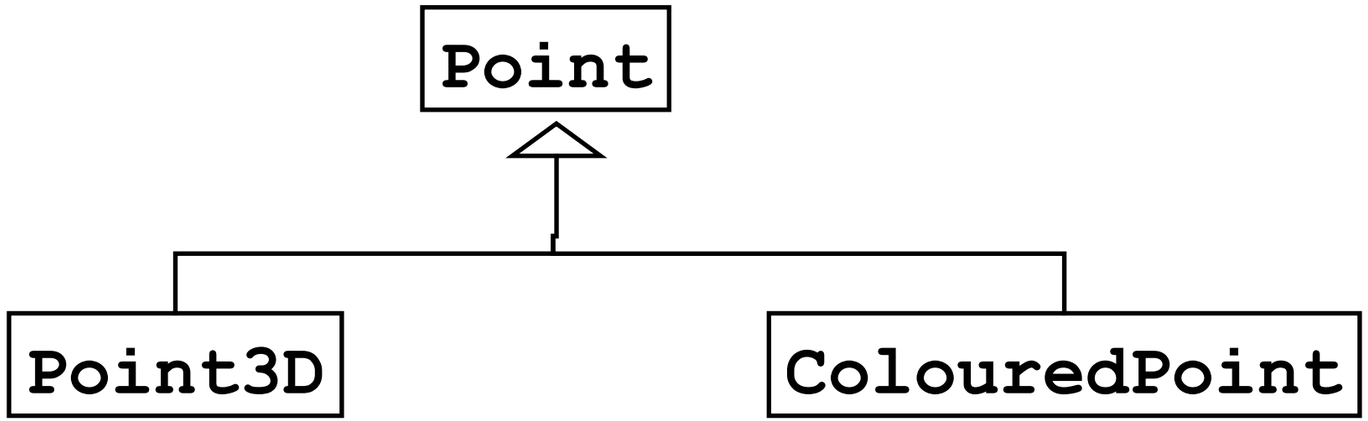
```
class Point { int x; int y; … }
class Point3D extends Point { int z; }
class ColouredPoint extends Point { int colour; }
class ColouredPoint3D extends ColouredPoint { int z; }

void move(Point p) { … }
void paint(ColouredPoint cp) { … }

ColouredPoint3D cp3= new ColouredPoint3D(…);
Point3D p3= new Point3D(…);
```

A) `move(cp3);` B) `move(p3);`

C) `paint(p3);`

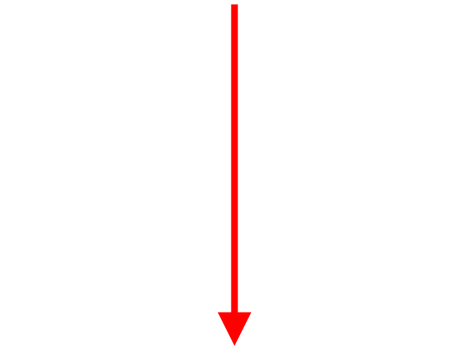
Up Casting



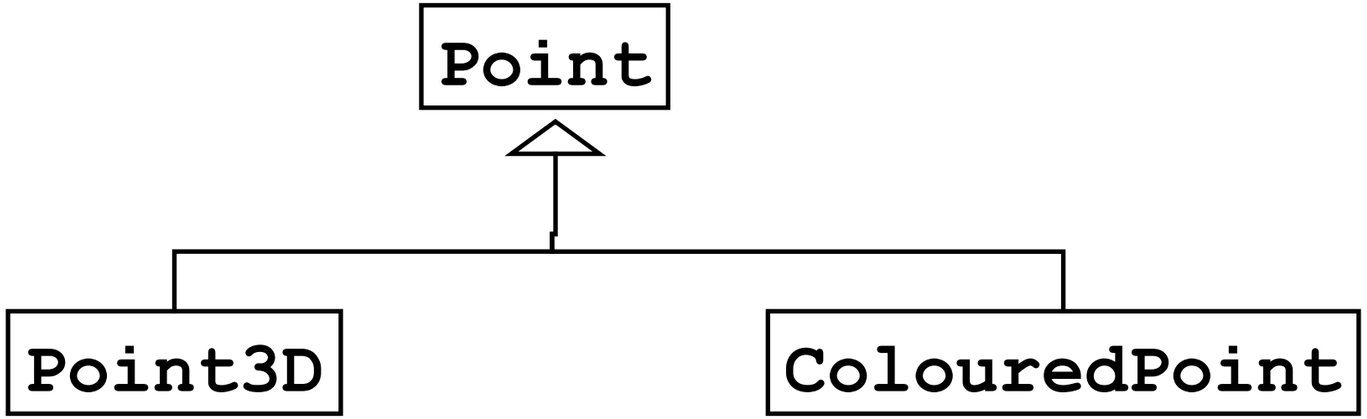



```
Point3D doSomething() { … }


Point p= doSomething();
```

Down Casting

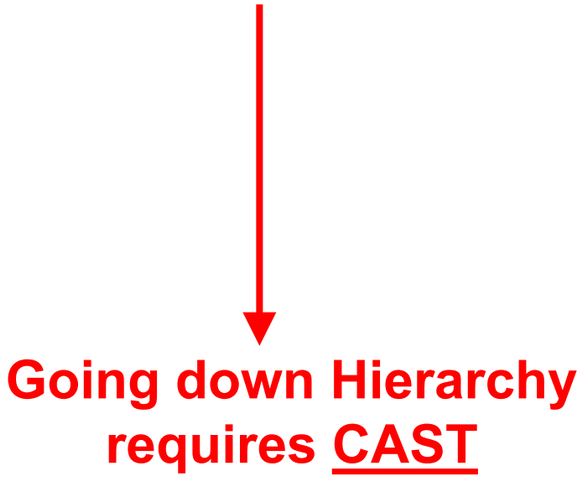


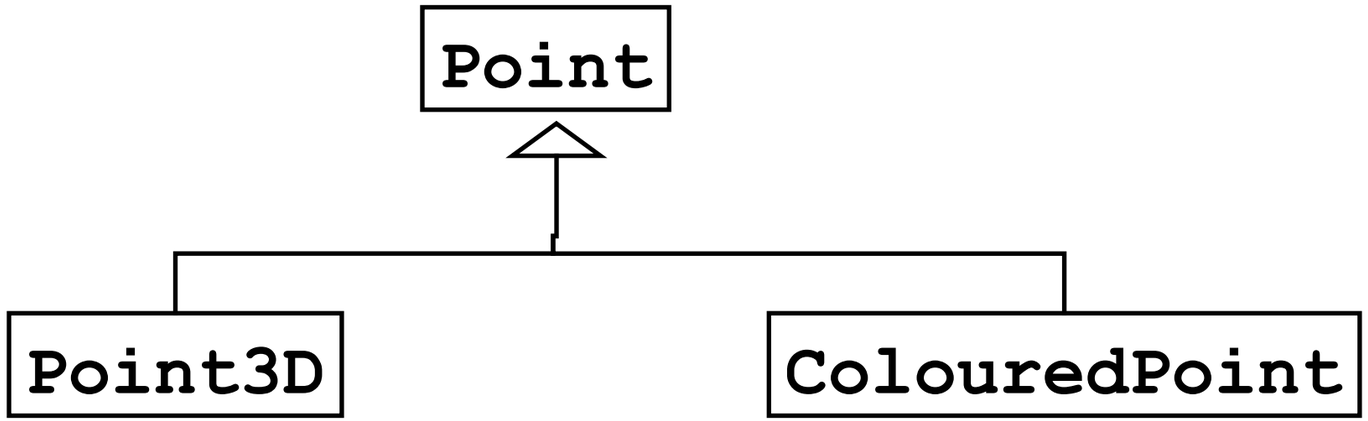
Going down
Hierarchy not OK




```
Point doSomething() { … }

Point3D p= doSomething();
```

Down Casting






```
Point doSomething() { … }

Point3D p= (Point3D) doSomething();
```

- Will throw exception if not Point3D!

Quiz – what gets printed?

```
class Person { … }

class Car {
  void shutDoor(Person p){ System.out.println("click"); }
}
class BigCar extends Car {
  void shutDoor(Person p) { System.out.println("BANG!"); }
}


...
Person    jim= new Person();
Car      small= new Car();
BigCar  large= new BigCar();
Car disguised= new BigCar();
small.shutDoor(jim);
large.shutDoor(jim);
disguised.shutDoor(jim);
```

A)
“click”
“click”
“click”

B)
“click”
“BANG!”
“BANG!”

C)
“click”
“click”
“BANG!”

D)
“click”
“BANG!”
“click”

Quiz – what gets printed?

```
class Person { … }

class Car {
  void shutDoor(Person p){ System.out.println("click"); }
}
class BigCar extends Car {
  void shutDoor(Person p) { System.out.println("BANG!"); }
}


...
Person    jim= new Person();
Car     small= new Car();
BigCar  large= new BigCar();
Car disguised= new BigCar();
small.shutDoor(jim);
large.shutDoor(jim);
disguised.shutDoor(jim);
```

A)
“click”
“click”
“click”

B)
“click”
“BANG!”
“BANG!”

C)
“click”
“click”
“BANG!”

D)
“click”
“BANG!”
“click”

No change!

- The static type did not changed the behavior
- Sadly, this is not the case for the ugly parts of Java
- Ugly == the static type changes the behavior


Subtyping
Coercion

Primitive types do not have normal subtyping/casting

- Numbers (widening coercion)
  - less precise **<:** more precise
  - **int <: float**
  - **float <: double**
- Narrowing conversions
  - Requires explicit coercion

```
What is the correct code?
--TEST1--
(A)
  float a= 3f;
  int b= a;
---------------
(B)
  int a= 3;
  float b= a;
---------------
```

Same syntax of a cast
but different semantic!

Primitive types do not have normal subtyping/casting

- Numbers (widening coercion)
  - less precise **<:** more precise
  - **int <: float**
  - **float <: double**
- Narrowing conversions
  - Requires explicit coercion

```
What is the correct code?
--TEST1--
(A)Explicit coercion
  float a= 3f;
  int b= (int)a;
---------------
(B)Correct
  int a= 3;
  float b= a;
---------------
```

Same syntax of a cast
but different semantic!

Primitive types do not have normal subtyping/casting

- Numbers (widening coercion)
  - less precise **<:** more precise
  - **int <: float**
  - **float <: double**
- Narrowing conversions
  - Requires explicit coercion

Same syntax of a cast
but different semantic!

```
What is the correct code?
--TEST1--
(A)Explicit coercion
  float a= 3f;
  int b= (int)a;
---------------
(B)Correct
  int a= 3;
  float b= a;
---------------
--TEST2--
(A)
  double a= 3d;
  float b= a;
---------------
(B)
  float a= 3f;
  double b= a;
```

Primitive types do not have normal subtyping/casting

- Numbers (widening coercion)
  - less precise **<:** more precise
  - **int <: float**
  - **float <: double**
- Narrowing conversions
  - Requires explicit coercion

Same syntax of a cast
but different semantic!

```
What is the correct code?
--TEST1--
(A)Explicit coercion
  float a= 3f;
  int b= (int)a;
---------------
(B)Correct
  int a= 3;
  float b= a;
---------------
--TEST2--
(A)Explicit coercion
  double a= 3d;
  float b= (float)a;
---------------
(B)Correct
  float a= 3f;
  double b= a;
```

Type coercion

- `1 + 2.0 =`
- `1 / 2 =`
- `1 / 2.0 =`
- `int x= 1/ 2.0;`

Type coercion

- `1 + 2.0 = 3.0 --> 3.0f or 3.0d ?`
- `1 / 2 =`
- `1 / 2.0 =`
- `int x= 1/ 2.0;`

Type coercion

- 1 + 2.0 = 3.0 --> 3.0f or 3.0d ?
- **1 / 2 = 0**
- **1 / 2.0 =**
- **int x= 1/ 2.0;**

Type coercion

- 1 + 2.0 = 3.0 --> 3.0f or 3.0d ?
- 1 / 2 = 0
- **1 / 2.0 = 0.5 = 1/2d = 0.5d**
- **int x= 1/ 2.0;**

Type coercion

- 1 + 2.0 = 3.0 --> 3.0f or 3.0d ?
- 1 / 2 = 0
- 1 / 2.0 = 0.5 = 1/2d = 0.5d
- **int x= 1/ 2.0;//not compile**

- **int x= (int)(1/2.0); --> x == 0**

Type coercion

- `1 + 2.0 = 3.0 --> 3.0f or 3.0d ?`
- `1 / 2 = 0`
- `1 / 2.0 = 0.5 = 1/2d = 0.5d`
- `int x= 1/ 2.0;//not compile`

- `int x= (int)(1/2.0); --> x == 0`

- Simplify your life: every time you write a numeric literal, if you have any doubt, or if you want to make your code easier to read, use f or d.
- 1/2f do what you usually expect.
- 1/2f == 0.5f
- 1/2 != 1/2f 1/2f != 1/2d

Type coercion

(T)e
example, what is true, what is false?
((int)2.5f) == 2

(A) for true
(B) for false

Type coercion

(A) for true
(B) for false

(T)e
example, what is true, what is false?
((int)2.5f) == 2 true

Type coercion

(A) for true
(B) for false

(T)e
example, what is true, what is false?
((int)2.5f) == 2 true
(int)1.5f + 2.5f == 4

Type coercion

(A) for true
(B) for false

(T)e
example, what is true, what is false?

((int)2.5f) == 2	true	
(int)1.5f + 2.5f == 4	false	… == 3.5f

Type coercion

(A) for true
(B) for false

(T)e
example, what is true, what is false?

((int)2.5f) == 2	true
(int)1.5f + 2.5f == 4	false
(int)(1.5f + 2.5f) == 4	

Type coercion

(A) for true
(B) for false

(T)e
example, what is true, what is false?

((int)2.5f) == 2	true
(int)1.5f + 2.5f == 4	false
(int)(1.5f + 2.5f) == 4	true ?

Type coercion

(A) for true
(B) for false

(T)e
example, what is true, what is false?

((int)2.5f) == 2	true
(int)1.5f + 2.5f == 4	false
(int)(1.5f + 2.5f) == 4	true ?

(int)(100000000.5f + 200000000.5f) == 300000001?

Type coercion

(A) for true
(B) for false

(T)e
example, what is true, what is false?
((int)2.5f) == 2 true
(int)1.5f + 2.5f == 4 false
(int)(1.5f + 2.5f) == 4 true ?
(int)(100000000.5f + 200000000.5f) == 300000001?

false?

Type coercion

Coercion creates a new number from another one.
No modification is applied to the original number
No modification is applied to the variable/field containing the original number

Another number is created.
It is “similar” to the original one, but in the new representation.

Type coercion != Class Cast

Comparision: Type coercion V.S. Class cast

Coercion creates a new number.

Class Cast is just a check. Does not create anything.

Coercion makes another entity (the new number), somehow similar to the original one.

Class Cast evaluates in the original object.

(float)(int)2.5f != 2.5f

(Point)(Object)p == p (with Point p=new Point();)

Note: (Object)p == p

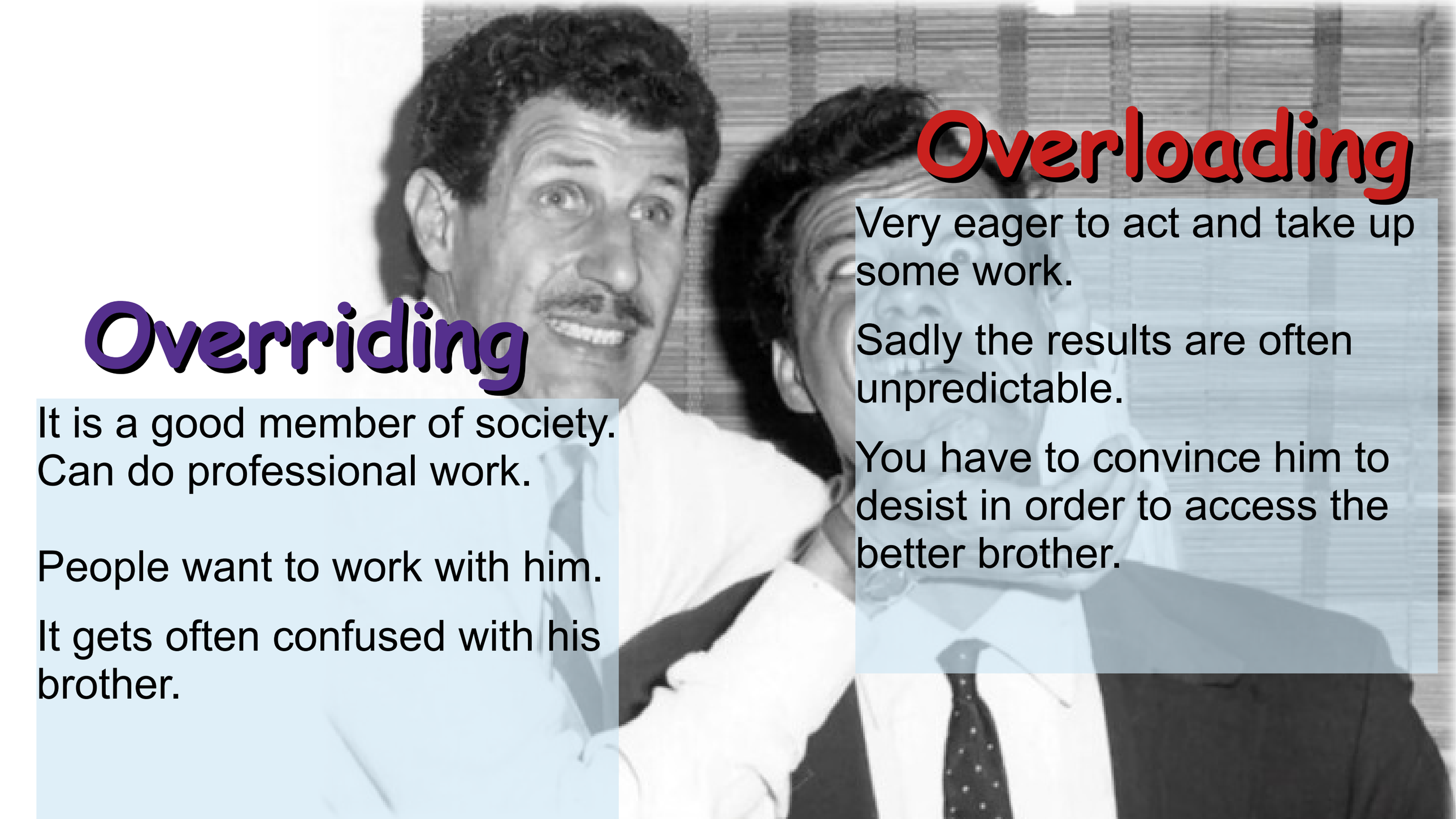

Overloading
Overriding
The two brothers

Overriding

It is a good member of society.
Can do professional work.

People want to work with him.

It gets often confused with his brother.



Overriding

It is a good member of society.
Can do professional work.

People want to work with him.

It gets often confused with his brother.

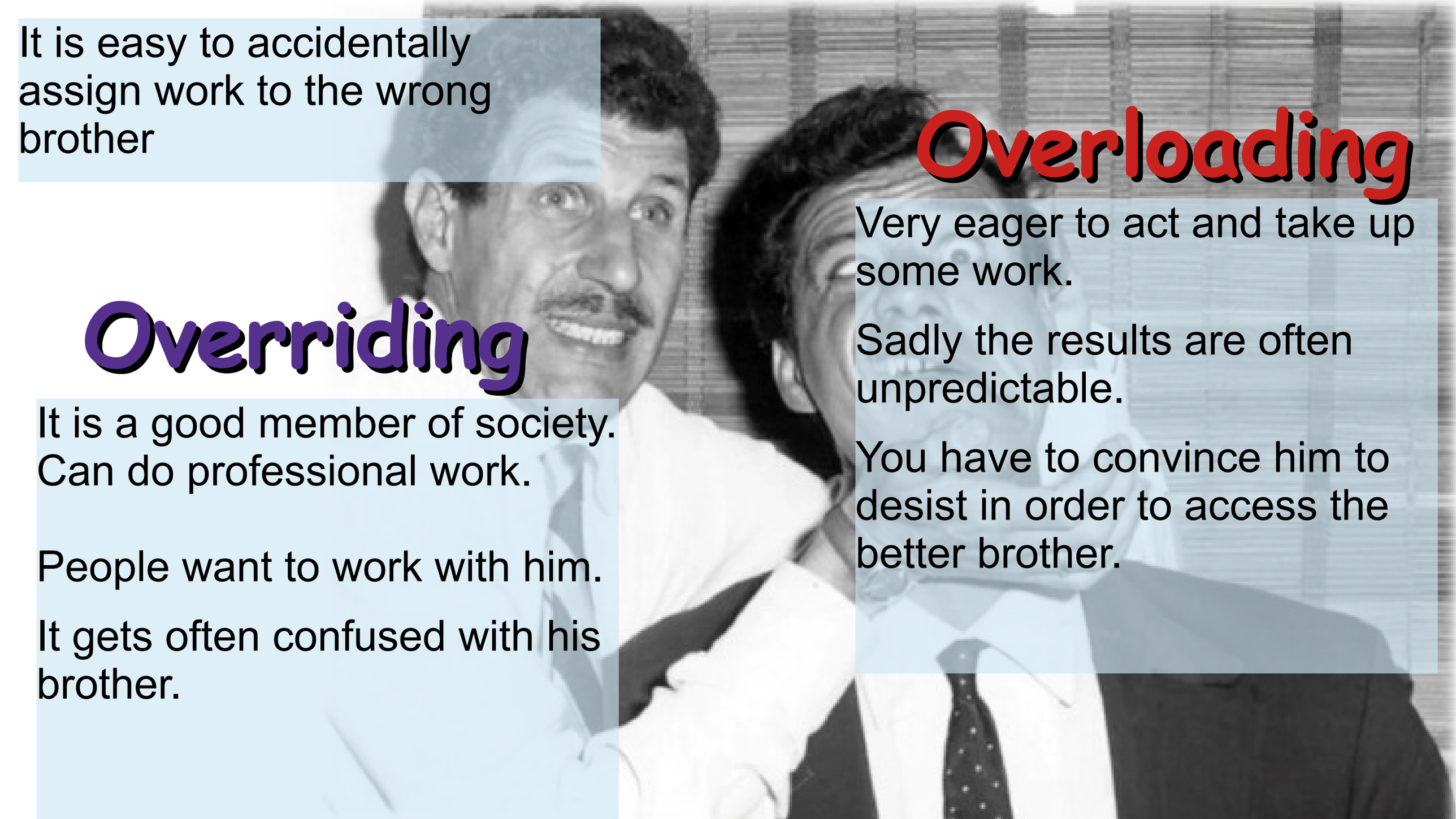
Overloading

Very eager to act and take up some work.

Sadly the results are often unpredictable.

You have to convince him to desist in order to access the better brother.

It is easy to accidentally assign work to the wrong brother



Overriding

It is a good member of society. Can do professional work.

People want to work with him.

It gets often confused with his brother.

Overloading

Very eager to act and take up some work.

Sadly the results are often unpredictable.

You have to convince him to desist in order to access the better brother.

Method overloading

- Two different methods can have same name,
  but different parameter types

```
class Car {
  void shutDoor(Person p) { System.out.println("click"); }
  void shutDoor(StrongPerson s) { System.out.println("Bang!"); }
}
```

  - Unfortunately, it is **dangerous** to do this
    - Ok, when different number of parameters

overloading/overriding

- Two methods in overloading are **two different methods**
  - The hidden real method name == method descriptor:
    - Method name + fully qualified name of all the arguments

- Two methods in overriding are **the same method** with two different bodies
  - The specific body is selected by dynamic dispatch at run time.

Quiz – what gets printed?

```
class Person { … }
class StrongPerson extends Person { … }

class Car {
 void shutDoor(Person p) {
  System.out.println("click");
 }
 void shutDoor(StrongPerson s) {
  System.out.println("BANG!");
}}

Car             c= new Car();
Person        jim= new StrongPerson();
StrongPerson henry= new StrongPerson();
c.shutDoor(jim);
c.shutDoor(henry);
```

A)

```
"click"
"click"
```

B)

```
"BANG!"
"BANG!"
```

C)

```
"click"
"BANG!"
```

Quiz – what gets printed?

```
class Person { … }
class StrongPerson extends Person { … }

class Car {
 void shutDoor(Person p) {
  System.out.println("click");
 }
 void shutDoor(StrongPerson s) {
  System.out.println("BANG!");
}}

Car             c= new Car();
Person        jim= new StrongPerson();
StrongPerson henry= new StrongPerson();
c.shutDoor(jim);
c.shutDoor(henry);
```

A)

```
"click"
"click"
```

B)

```
"BANG!"
"BANG!"
```

C)

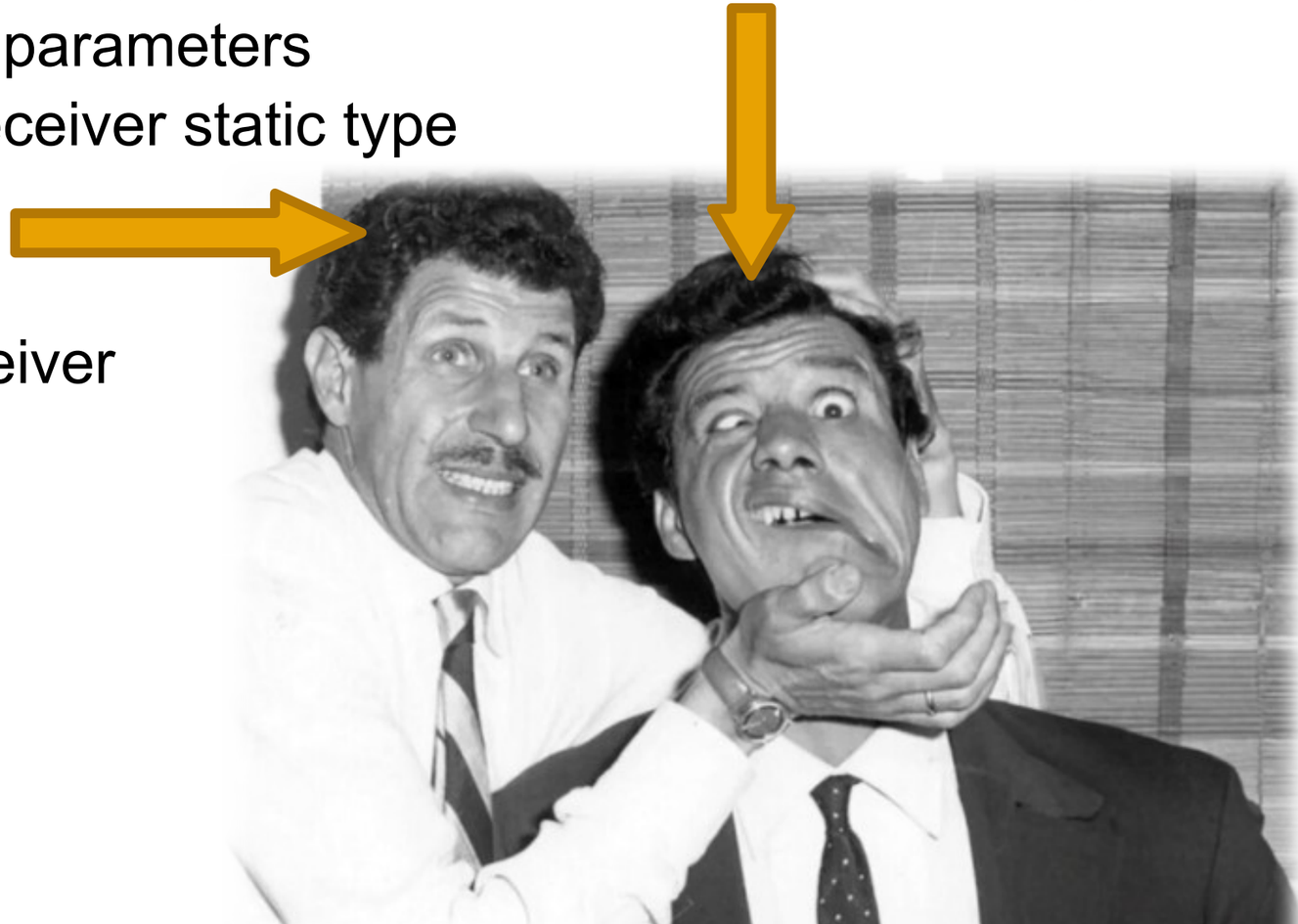
```
"click"
"BANG!"
```

Method dispatch

```
x.aMethod(y);
```
`x` is the receiver

`y` is the parameter

- Method dispatch:
  - Two phases – compile time and runtime
- Static checking phase (**overloading** resolution): //That is, we have to handle the ugly first
  - Based on static types of receiver and parameters
  - Only visible methods defined in the receiver static type
- Dynamic dispatch (**overriding** resolution):
  - Selection of method at runtime
  - Dynamic choice depends only on receiver



Dispatch 1: static checking

- Identify ***static*** type of receiver
- Find methods that could possibly apply
  - Correct name (also search in super-types)
  - Visible
  - Correct number of arguments
  - Argument types that could apply
    - By looking at ***Static types***
    - Take into account subtyping and coercion
- Choose the most specific method (overloading)

Dispatch 1: static checking - Method descriptor

Method descriptor == Method name and static types of arguments.

Not return types

As result of this phase you have a

method descriptor

The method ***descriptor*** is chosen statically

It is ‘as if’ Javac during compilation were replacing the method names with the more pedantic method descriptors.

Dispatch 1: static checking

What is a method descriptor?

As an example **m(Foo,Bar,int)**

- It is a methodName followed by the statically declared type of the parameters.
- Descriptors are the “overloaded resolved” version of a method name.
- If you have found m(Foo,Bar,int) as result of phase 1, it means that some class define a method **m** with **Foo**, **Bar** and **int** as parameter types.

Dispatch 2: dynamic dispatch

To determine which method is called at runtime:

a: Identify ***dynamic*** type of receiver

b: Check for the method implementation

using the method *descriptor* **(method descriptor used static types of arguments)**

If the method descriptor is not there, then look in super type(s) and then its super type(s) until match

Since the descriptor is from phase 1, you will find one match.

Method resolution

Big quiz day!!

Quiz

```
class Cat {
 String whatAmI(){ return “I’m a Cat!”; }
 void print(){ System.out.println(whatAmI()); }
}
class Kitten extends Cat {
 String whatAmI(){ return “I’m a Kitten!”; }
}
Cat gypsy= new Cat();
Cat spike= new Kitten();
gypsy.print();
spike.print();
```

A)
“I’m a Kitten!”
“I’m a kitten!”

B)
“I’m a Cat!”
“I’m a Kitten!”

C)
“I’m a Cat!”
“I’m a Cat!”

Quiz

```
class Cat {
 String whatAmI(){ return “I’m a Cat!”; }
 void print(){ System.out.println(whatAmI()); }
}
class Kitten extends Cat {
 String whatAmI(){ return “I’m a Kitten!”; }
}
Cat gypsy= new Cat();
Cat spike= new Kitten();
gypsy.print();
spike.print();
```

A)
“I’m a Kitten!”
“I’m a kitten!”

B)
“I’m a Cat!”
“I’m a Kitten!”

C)
“I’m a Cat!”
“I’m a Cat!”

Quiz

```
class Cat {
 public void action(Cat c){ System.out.println(1); }
 public void action(Kitten c){ System.out.println(2); }
}

class Kitten extends Cat { }

Cat    gypsy= new Cat();
Cat    spike= new Kitten();
Kitten teddy= new Kitten();
gypsy.action(spike);
spike.action(teddy);
teddy.action(teddy);
```

A)
```
1
2
2
```

B)
```
1
1
2
```

Quiz

```
class Cat {
 public void action(Cat c){ System.out.println(1); }
 public void action(Kitten c){ System.out.println(2); }
}

class Kitten extends Cat { }

Cat    gypsy= new Cat();
Cat    spike= new Kitten();
Kitten teddy= new Kitten();
gypsy.action(spike);
spike.action(teddy);
teddy.action(teddy);
```

A)
1
2
2

B)
1
1
2

Quiz

```
class Cat {
 public void action(Cat c){ System.out.println(1); }
}

class Kitten extends Cat {
 public void action(Kitten k){ System.out.println(2); }
}

Cat    gypsy= new Cat();
Cat    spike= new Kitten();
Kitten teddy= new Kitten();
gypsy.action(teddy);
spike.action(teddy);
teddy.action(teddy);
```

A)
```
1
2
2
```

B)
```
1
1
2
```

Quiz

```
class Cat {
 public void action(Cat c){ System.out.println(1); }
}

class Kitten extends Cat {
 public void action(Kitten k){ System.out.println(2); }
}

Cat    gypsy= new Cat();
Cat    spike= new Kitten();
Kitten teddy= new Kitten();
gypsy.action(teddy);
spike.action(teddy);
teddy.action(teddy);
```

A)

```
1
2
2
```

B)

```
1
1
2
```

Quiz

```
class Cat {
 public void action(Kitten c){ System.out.println(1); }
}

class Kitten extends Cat {
 public void action(Cat k){ System.out.println(2); }
}

Cat    gypsy= new Cat();
Cat    spike= new Kitten();
Kitten teddy= new Kitten();
gypsy.action(teddy);
spike.action(teddy);
teddy.action(teddy);
```

A)
```
1
1
2
```

B)
```
1
1
1
```

Quiz

```
class Cat {
 public void action(Kitten c){ System.out.println(1); }
}

class Kitten extends Cat {
 public void action(Cat k){ System.out.println(2); }
}

Cat    gypsy= new Cat();
Cat    spike= new Kitten();
Kitten teddy= new Kitten();
gypsy.action(teddy);
spike.action(teddy);
teddy.action(teddy);
```

A)
```
1
1
2
```

B)
```
1
1
1
```

Quiz

```
class Cat {
  public void action(Cat c, Kitten k){ System.out.println(1); }
}

class Kitten extends Cat {
  public void action(Kitten k, Cat c){ System.out.println(2); }
}

Cat    gypsy= new Cat();
Cat    spike= new Kitten();
Kitten teddy= new Kitten();
gypsy.action(spike,teddy);
spike.action(teddy,spike);
teddy.action(teddy,teddy);
```

A) 1, 1, 2

B) compilation error

C) 1, 2, 2

Quiz

```
class Cat {
  public void action(Cat c, Kitten k){ System.out.println(1); }
}

class Kitten extends Cat {
  public void action(Kitten k, Cat c){ System.out.println(2); }
}

Cat    gypsy= new Cat();
Cat    spike= new Kitten();
Kitten teddy= new Kitten();
gypsy.action(spike,teddy);
spike.action(teddy,spike);
teddy.action(teddy,teddy);
```

A) 1, 1, 2 

B) compilation error 

C) 1, 2, 2 

Forcing overloading resolution

Just add casts!!

Just cast the actual parameters
to the specific supertype needed
to select the desired overloaded method

Another ugly fact

- Order of initialization can be very relevant
- It is the source of many subtle bugs

Order of execution:

- 1- Static initializers, but only the first time.
- 2- Super call and recursive super type initialization.
- 3- Field initializers and instance initialization block (in the order they appear in the code)
- 4- The rest of the constructor

Order of execution!

```
class Base{
  Base(){ System.out.println("Base"); } //body of super constructor
}

class Heir extends Base{
  int zero= doInit(0);//in place initialization expression

  { System.out.println("initialization block"); }//initialization block

  static { System.out.println("static init block"); }//static initialization block

  int doInit(int i){ System.out.println("init "+i); return i; }

  public Heir(){ System.out.println("Heir"); }//constructor body

  int one= doInit(1);

  { System.out.println("initialization block 2"); }
}

...
new Heir();//Question: What this line outputs?
```

```
-1
-2
-3
-4
-5
-6
-7
```

Order of execution!

```
class Base{
  Base(){ System.out.println("Base"); } //body of super constructor
}

class Heir extends Base{
  int zero= doInit(0);//in place initialization expression

  { System.out.println("initialization block"); }//initialization block

  static { System.out.println("static init block"); }//static initialization block

  int doInit(int i){ System.out.println("init "+i); return i; }

  public Heir(){ System.out.println("Heir"); }//constructor body

  int one= doInit(1);

  { System.out.println("initialization block 2"); }
}

...
new Heir();//Question: What this line outputs?
```

```
-1 static init block
-2
-3
-4
-5
-6
-7
```

Order of execution!

```
class Base{
  Base(){ System.out.println("Base"); } //body of super constructor
}

class Heir extends Base{
  int zero= doInit(0);//in place initialization expression

  { System.out.println("initialization block"); }//initialization block

  static { System.out.println("static init block"); }//static initialization block

  int doInit(int i){ System.out.println("init "+i); return i; }

  public Heir(){ System.out.println("Heir"); }//constructor body

  int one= doInit(1);

  { System.out.println("initialization block 2"); }
}

...
new Heir();//Question: What this line outputs?
```

```
-1 static init block
-2 Base
-3
-4
-5
-6
-7
```

Order of execution!

```
class Base{
  Base(){ System.out.println("Base"); } //body of super constructor
}

class Heir extends Base{
  int zero= doInit(0);//in place initialization expression

  { System.out.println("initialization block"); }//initialization block

  static { System.out.println("static init block"); }//static initialization block

  int doInit(int i){ System.out.println("init "+i); return i; }

  public Heir(){ System.out.println("Heir"); }//constructor body

  int one= doInit(1);

  { System.out.println("initialization block 2"); }
}

...
new Heir();//Question: What this line outputs?
```

```
-1 static init block
-2 Base
-3 init 0
-4
-5
-6
-7
```

Order of execution!

```
class Base{
  Base(){ System.out.println("Base"); } //body of super constructor
}

class Heir extends Base{
  int zero= doInit(0);//in place initialization expression

  { System.out.println("initialization block"); }//initialization block

  static { System.out.println("static init block"); }//static initialization block

  int doInit(int i){ System.out.println("init "+i); return i; }

  public Heir(){ System.out.println("Heir"); }//constructor body

  int one= doInit(1);

  { System.out.println("initialization block 2"); }
}

...
new Heir();//Question: What this line outputs?
```

```
-1 static init block
-2 Base
-3 init 0
-4 initialization block
-5
-6
-7
```

Order of execution!

```
class Base{
  Base(){ System.out.println("Base"); } //body of super constructor
}

class Heir extends Base{
  int zero= doInit(0);//in place initialization expression

  { System.out.println("initialization block"); }//initialization block

  static { System.out.println("static init block"); }//static initialization block

  int doInit(int i){ System.out.println("init "+i); return i; }

  public Heir(){ System.out.println("Heir"); }//constructor body

  int one= doInit(1);

  { System.out.println("initialization block 2"); }
}

...
new Heir();//Question: What this line outputs?
```

```
-1 static init block
-2 Base
-3 init 0
-4 initialization block
-5 init 1
-6
-7
```

Order of execution!

```
class Base{
  Base(){ System.out.println("Base"); } //body of super constructor
}

class Heir extends Base{
  int zero= doInit(0);//in place initialization expression

  { System.out.println("initialization block"); }//initialization block

  static { System.out.println("static init block"); }//static initialization block

  int doInit(int i){ System.out.println("init "+i); return i; }

  public Heir(){ System.out.println("Heir"); }//constructor body

  int one= doInit(1);

  { System.out.println("initialization block 2"); }
}

...
new Heir();//Question: What this line outputs?
```

```
-1 static init block
-2 Base
-3 init 0
-4 initialization block
-5 init 1
-6 initialization block 2
-7
```

Order of execution!

```
class Base{
  Base(){ System.out.println("Base"); } //body of super constructor
}

class Heir extends Base{
  int zero= doInit(0);//in place initialization expression

  { System.out.println("initialization block"); }//initialization block

  static { System.out.println("static init block"); }//static initialization block

  int doInit(int i){ System.out.println("init "+i); return i; }

  public Heir(){ System.out.println("Heir"); }//constructor body

  int one= doInit(1);

  { System.out.println("initialization block 2"); }
}

...
new Heir();//Question: What this line outputs?
```

```
-1 static init block
-2 Base
-3 init 0
-4 initialization block
-5 init 1
-6 initialization block 2
-7 Heir
```